5.1. Curve fitting

5.1.1 Curve Fitting in Python……3
5.1.2 Interpreting curve fitting output……4
5.1.3 Visualize fit results……4
5.1.4 Least square method……5
5.1.5 Goodness of fit……7
5.1.6 Cautions in using R^2……10
5.1.7 Extension to non-linear models……11

Curve fitting is the process of finding the relationship between an independent variable (x) and a dependent variable (y) from experimental data sample. Curve fitting is one of the most common things you will do as an experimental scientist. You gather a set of data point, you visualize it (graph it), create a fit to some mathematical function and build a theoretical model.

Suppose you have performed and experiment and measured some data points y as a function of a variable quantity x. After the measurements, you have a set of data N points:

x_1	x_2	x_3	…	x_N
y_1	y_2	y_3	…	y_N

Now you want to find the functional relation between the variables $y = y(x)$ =?

The simplest function relating the two variables is the linear function

$$y = ax + b \qquad \text{(eq. 5.1)}$$

where the slope $'a'$ is a certain physical quantity and the intercept $'b'$ is some other physical quantity.

If you could find the values of a and b from the experiment, these values may contain estimates for useful quantities.

One way to determine a and b is to plot the data and draw a straight line through the points using a ruler in such way that the line will pass as close as possible in between the points.

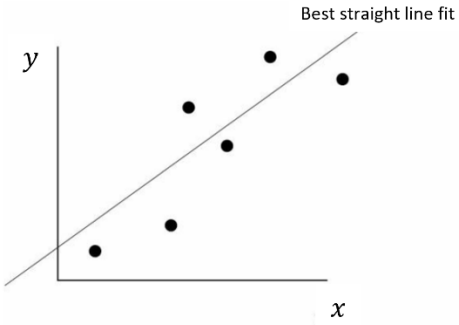


Fig 5.1 Best straight fit line to a collection of data points

A more accurate method is to perform curve fitting using Python. The curve_fit function uses the least squares method to find the line that passes best between the data point and gives as an output the values and uncertainties of a and b.

After the curve fitting procedure, you should provide:

1. Fitting parameters values (for example, in a linear model, the values of $\boldsymbol{a}$ and $\boldsymbol{b}$)
2. Error estimate of the parameters (for example $\boldsymbol{\delta a}$ and $\boldsymbol{\delta b}$).
3. Visualize data and fitted model with graph figure.
4. Measure of goodness of fit (for example $\boldsymbol{R^2}$).

5.1.1 Curve Fitting in Python

In the lab we will use the function: *curve_fit* , which is part of the *scipy.optimize* package. The syntax is:

popt,pcov = scipy.optimize.curve_fit(fit_func,x_data,y_data,sigma,guess)

popt - an array of the output fit parameters

pcov - an array of the covariance of the output fit parameters

fit_func - the function used to do the fit

sigma - the uncertainty associated with each data point (uncertainty bars, if needed)

guess - the initial guess input to the fit

Note: If the error (uncertainty) in the value of each measured data point is unknown or negligible, then no need to specify the sigma values. The function will use the default which is an vector of ones.

For example, suppose we have measured weight, w, of students as function of their height, h, and we want to know if there is a linear relationship and exactly what is the function $w(h)$?

Code to generate the data:

```
h = 1.65*np.ones(15)
delta = np.random.uniform(-0.15,0.15, size=len(h))
h = np.round((h + delta),2)
delta = np.random.uniform(-5,5, size=len(h))
w = -110 + 107*h + delta
```

Curve fitting code

```
12 #Load data and set variables for the data
13 data = np.loadtxt('whDAta.txt')
14 h = data[:,0] #height [m]
15 w = data[:,1] #weight [kg]
23 #define the function to be used in the fitting
24 def fit_func(h, b, a):
25     return a*h + b
26
27 ##add an uncertainty to each measurement point
28 uncertainty = np.empty(len(h))
29 uncertainty.fill(1)
30
31 #do the fit
32 from scipy.optimize import curve_fit
33 #popt,pcov = curve_fit(fit_func,h,w)
34 popt,pcov = curve_fit(fit_func,h,w,sigma=uncertainty)
```

We have assumed here uncertainty of 1 kg in the weight measurement (lines 28-29 in the code). If the uncertainty in your measurement is negligible, no need to write this part!

Output of the *curve_fit* function

```
In [95]: print(popt)
[-121.80931802  109.82915797]

In [96]: print(pcov)
[[ 503.57694712 -285.31455561]
 [-285.31455561  162.11054335]]
```

5.1.2 Interpreting curve fitting output

Parameters are represented in the result in the same order that the optimized function takes them. Since line is defined as *fit_fun(h, b, a),* **the *popt* vector contains the estimates in the order *[b, a].***

The *pcov* is a matrix that *curve_fit* returns is the covariance matrix of the estimated parameters.

- The diagonal elements are the square of the standard error of the parameters
- The non-diagonal elements show the relationship between the parameters

The diagonal elements are the variances. They tell us how likely the parameter is to vary, or how confident we are in the estimate. Hence,

```
#Curve fit result in a and b values
a = popt[1]
b = popt[0]
#determine the error for each parameter
da = np.sqrt(pcov[1,1])
db= np.sqrt(pcov[0,0])
```

Therefore, the parameters we are looking for are

$$\begin{cases} a = 110 \pm 22\ [kg/m] \\ b = -122 \pm 12\ [kg] \end{cases}$$

We set up the function for the fit to be:

$$\boldsymbol{y} = \boldsymbol{popt}[\mathbf{1}] \cdot \boldsymbol{x} + \boldsymbol{popt}[\mathbf{0}]$$

So, with the fit parameters, the function is:

$$y = 110h - 122$$

The off-diagonal elements of *pcov* are the covariances. Their meaning is actually hidden in the name: covariance – how the variables vary together.

5.1.3 Visualize fit results

Example code:

```
#now generate the line of the best fit:
#set up the height points for the full array
hT = np.arange(np.min(h),np.max(h),0.001)

#plot the data points and the fitted curve
plt.figure()
plt.errorbar(h,w,yerr=uncertainty,capsize=5, ls='none')
plt.plot(h,w,'bo' ,markersize=8)
plt.plot(hT,fit_func(hT,*popt),'k', linewidth=2)
plt.xlabel('height [m]',fontsize=14)
plt.ylabel('weight [kg]',fontsize=14)
plt.legend(['data','fit'])
```

The resulting figure is:

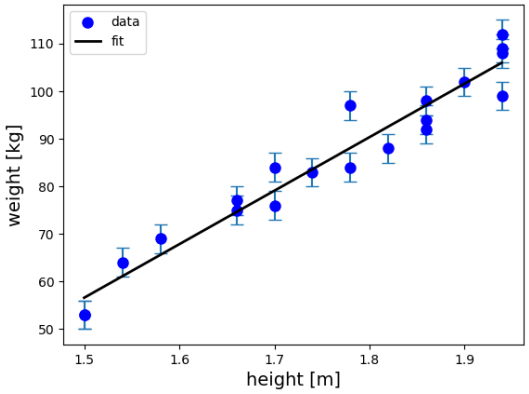


Fig 5.1.2 Data and fitted linear line

Uncertainty bars (or "error bars" are the I bar around the data points) on figure 5.1.1 are generated by line 52 in the example code below. Uncertainty bars showing the uncertainty range associated with each data point. That is, for repeated measurement of $y_i(x_i)$ will produce uncertainty which is the standard deviation in these values, s_y. These are the same values that are plugged into *sigma* option in the *curve_fit* function! If the uncertainties are large enough to be visible on the graph, you should show both vertical and horizontal uncertainty bars (If they are not large enough, you should mention this in your report, so we will not think you have forgotten them.)

5.1.4 Least square method

How does the **curve_fit** function find the best line to pass between the measured data points?

- The algorithm inside this function is looking for the values of the fit parameters (a and b) which are most likely to fit the data best.
- A line that fits the data best, is the line $(ax_i + b)$ the passes closest to the measured points (x_i, y_i)
- The distance between the data points and the fitted points is called **residuals:**

$$(residuals)_i = y_i - (ax_i + b)\hat{y}_i = a \cdot x_i + b \qquad \text{(eq. 5.2)}$$

- To achieve this the function $curve_fit$ uses a "minimizer". We will discuss the quantity which is minimized in a bit.

The probability of any one point, y_i being from the fit comes from normal distribution because all random errors are averaged as stated by the central limit theorem:

$$P_{a,b}(y_i) = \frac{1}{\sqrt{2\pi}s_y}\exp\left(-\frac{(y_i - ax_i - b)^2}{2s_y^2}\right) \qquad \text{(eq. 5.3)}$$

Where, y_i is the measured point, s_y is the uncertainty of y_i and a and b found from the fit.

For a set of N measurements of the dependent variable $y_1, y_2, \dots, y_N$, the full probability of obtaining these values is the product of the individual probabilities

$$P_{a,b}(y_1, y_2, \dots, y_n) = P_{a,b}(y_1)P_{a,b}(y_2) \dots P_{a,b}(y_n)$$
$$= \frac{1}{s_y^N} \exp\left(-\frac{1}{2}\sum_{i=1}^{N} \frac{(y_i - ax_i - b)^2}{s_y^2}\right) \qquad \text{(eq. 5.4)}$$

Let us define the chi-squared function,

$$\chi^2(a,b) = \sum_{i=1}^{N} \frac{(y_i - ax_i - b)^2}{s_y^2} \qquad \text{(eq. 5.5)}$$

Notice that the $(y_i - ax_i - b)$ are the residuals (the data point y_i minus the fit function $ax_i + b$). The square ensures this is always positive. Dividing the residuals by the standard deviation, s, tells us how many standard deviations the test data is away from the fit at that x_i.

It is obvious that minimization of the sum of squared residuals will results in greater probability $\boldsymbol{P_{a,b}(y_1, y_2, \dots, y_N)}$, thus greater agreement between the data and the fitted line.

A line that fits the data "best" will be the one for which the sum of prediction errors over all measurements n, one for each observed data point, are minimal compared to all the other possible lines we could have drawn between the measured points. In other words, **the curve fitting algorithm looks for values of a, b such that it approximates a value $\hat{y}_i$ for a certain measurement x_i, such that $\hat{y}_i$ will be as "close as possible" to the measured y_i.**

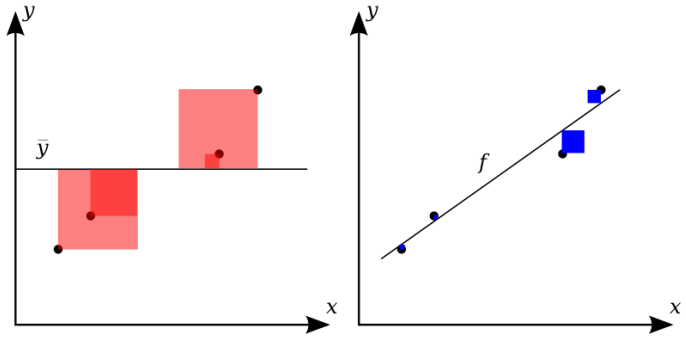


Fig 5.1.3 Determination of a best fit line by the method of least squares

This distance criterion is the "**minimal least squares method**". That is, the algorithm minimizes the "squared sum of residuals", or SSE (Fig. 5.1.3):

$$SSE = \sum (residuals)^2 = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{N} (y_i - ax_i - b)^2 \qquad \text{(eq. 5.6)}$$

***SSE* is measure of how far off our model's predictions are from the observed values. A value of 0 indicates that all predications are spot on. A non-zero value indicates errors. We expect a non-zero value. Why? Because there are always random errors.**

The better the linear regression (on the right) fits the data in comparison to the simple average (on the left graph), the closer the value of R is to 1 (see goodness of fit). The areas of the blue squares represent the squared residuals with respect to the linear regression. The areas of the red squares represent the squared residuals with respect to the average value.

Note: Suppose we are fitting linear model to data with errors in both coordinate, s_y are the errors in the y data points and s_x are the errors in the x data points. One solution is to modify the chi-squared function to include both errors:

$$\chi^2(a,b) = \sum_{i=1}^{N} \frac{(y_i - ax_i - b)^2}{s_y^2 + a^2 s_x^2} \qquad \text{(eq. 5.7)}$$

This procedure, however, is not symmetric – it minimizes the deviation in y, not necessarily in x.

5.1.5 Goodness of fit

Goodness of fit gives us a measure to quantify how well the predicted model approximated the model. Using *curve_fit* you can find the line that best fits the data (in the sense of least squares), but the best line may still not be a good fit! Knowing that you have the best solution (best fit, according to some criterium) does not necessarily mean that it is a good solution (therefore you need to check the goodness of the (best) fit). The decision if the fit is good or not is very challenging sometimes and many statistical model have been developed to address this question. Here we will mention 3 methods, each has strengths and weaknesses, so you need to use them wisely!

1. Coefficient of determination: r-squared

First, we want to check is whether the data could be adequately fitted by a linear model although the random errors. **If $r^2 \to 1$ this means that the relation is more linear. As rule of thumb, values of r^2 above 0.9 means that the relation is strongly linear!**

For simplicity,

$$r^2 = 1 - \frac{SSE}{SST} \qquad \text{(eq. 5.9)}$$

Where, SST is a measure of the variance in the target variable. It is measured simply as the sum of the squared difference between each observation and the mean.

$$SST = \sum_{i=1}^{N} (y_i - \bar{y})^2 \qquad \text{(eq. 5.10)}$$

where $\bar{y}$ is the mean of the measured data points,

$$\bar{y} = \sum_{i=1}^{N} \mathrm{y_i} \qquad \text{(eq. 5.11)}$$

The interpretation of r^2 follows from examining eq. 5.6:

If the linear model perfectly fits the measured data, then $\hat{y}_i = y_i$. Then, $SSE = 0$, and $r^2 = 1$. The interpretation here is that the linear model fits the data perfectly: The more the data is linear, $r^2 \to 1$.

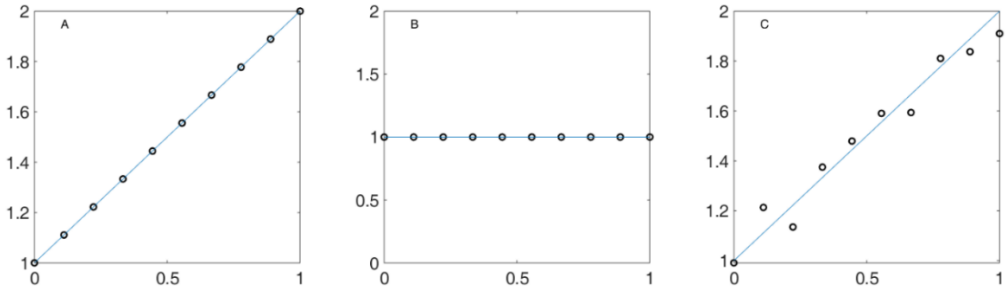


Figure 5.1.4 r-squared examples. A. $r^2 = 1$. B. $r^2 = 0$. C. $0 < r^2 < 1$

The opposite limit is when variations in x does not imply variations in y, in other words when for different x values we get the same y values. This happens when y is not a function of x, when the estimated regression line is perfectly horizontal, and the linear model fails. In this case $SSE = SST = 0$ and $r^2 = 0$ (figure 5.1.4b). In an experiment, because of random errors, perfect fit ($r^2 = 1$) is not possible.

Example code:

```
#calculate the residuals
residuals = w-fit_func(h,*popt)
#calculate the residuals sums of squares
ss_res = np.sum(residuals**2)
#calculate the total sum of squares
ss_tot = np.sum((w-np.mean(w))**2)
r_squared = 1 - (ss_res / ss_tot)
print('r_squared',r_squared)
```

2. Plot of residuals

Residual plots show the difference between actual and predicted values. If the points in a residual plot are randomly dispersed around the horizontal axis, a linear model is appropriate for the data; otherwise, a non-linear model is more appropriate. Plotting the residuals can give an important information regarding the goodness of fit. The residuals can be plotted to look for trends and see if the fit function is appropriate

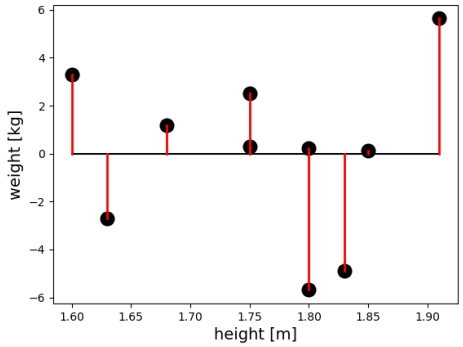


Fig. 5.1.4 Residual plot

As we can see, the points are randomly distributed around 0, meaning linear regression is an appropriate model to predict our data. If the residual plot presents a curvature (see figure 5.1.4), the linear assumption is incorrect. In this case, a non-linear function will be more suitable to predict the data.

Example code:

```
#make a line at zero
zero_line = np.zeros(len(h))
#plot residuals
plt.figure()
plt.plot(h,residuals,'ko',markersize=12)
plt.plot(h,zero_line,'k')
# now add individual line for each point
for i in range(len(h)):
    lineXdata = (h[i], h[i]) # same X
    lineYdata = (0, residuals[i]) # different Y
    plt.plot(lineXdata, lineYdata,'r', linewidth=2)
plt.xlabel('height [m]',fontsize=14)
plt.ylabel('weight [kg]',fontsize=14)
```

3. Reduced Chi-squared test for goodness of fit

In the discussion of least-square-method, we have defined the chi-squared function

$$\chi^2(a,b) = \sum_{i=1}^{N} \frac{(y_i - ax_i - b)^2}{s_y^2} \qquad \text{(eq. 5.12)}$$

Stating that the least-square method looking for the values of a and b which minimize the chi-squared function and therefore maximize the likelihood that our model line ($\hat{y}_i = ax_i - b$) will pass close as possible to the experimental points y_i given the uncertainty s_y in the measured points.

This chi-squared function can then be used as a "goodness of fit" test:

If this function is a good approximation, then the residual will be within one standard deviation, so this will sum to approximately $\boldsymbol{N}$.

Let us define the reduces chi-squared function,

$$\frac{\chi^2}{DOF} = \frac{\chi^2}{n-p} \qquad \text{(eq. 5.13)}$$

Where DOF is **number of degrees of freedom** of the experiment to determine the fit quality. The number of DOF is the number of data points in the sample minus the number of parameters in the fit:

$$DOF = N - p \qquad \text{(eq. 5.14)}$$

For a sample with 20 data points and a linear fit (2 parameters), $DOF = 18$.

If $\frac{\chi^2}{DOF} \sim \mathbf{1}$, this is a good fit. The sum of residuals is within one standard deviation!

If $\frac{\chi^2}{DOF} \gg \mathbf{1}$, either using inappropriate model or uncertainty bars on data ($\boldsymbol{s_y}$) are too small!

If $\frac{\chi^2}{DOF} \ll \mathbf{1}$, too good to be true, be suspicious! Probably the uncertainty bars are too large!

5.1.6 Cautions in using R^2

A caution that applies to R^2, as to other statistical descriptions of correlation and association is that "correlation does not imply causation." In other words, while correlations may sometimes provide valuable clues in uncovering causal relationships among variables, a non-zero estimated correlation between two variables is not, on its own, evidence that changing the value of one variable would result in changes in the values of other variables. **For example, the decreasing number of pirates is correlated with the increase of global average temperature. But decrease in the number of pirates does not cause the global temperature to rise.**

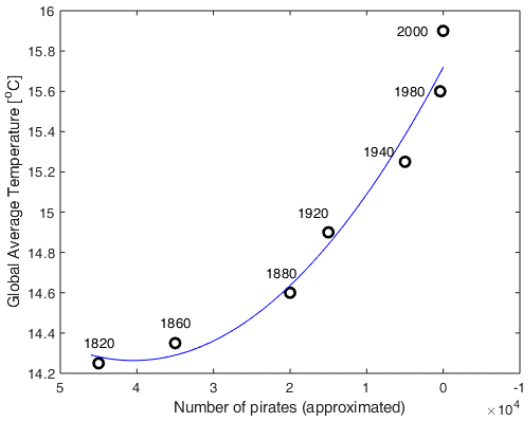


Figure 5.1.5 More pirates less global warming? Correlation does not mean causation.
https://kaiserscience.wordpress.com/physics/the-scientific-method/correlation-and-causation/

Notice that on its own, R^2, is not a good criterion for assessing the goodness of fit. For example, in all the given plots in figure 5.1.6 will result with the same value of R^2. While, only for the first plot in figure 5.1.6, the model fits well to the data. Therefore, the value of R^2 has to be considered along with additional test, for example plot of the residuals.

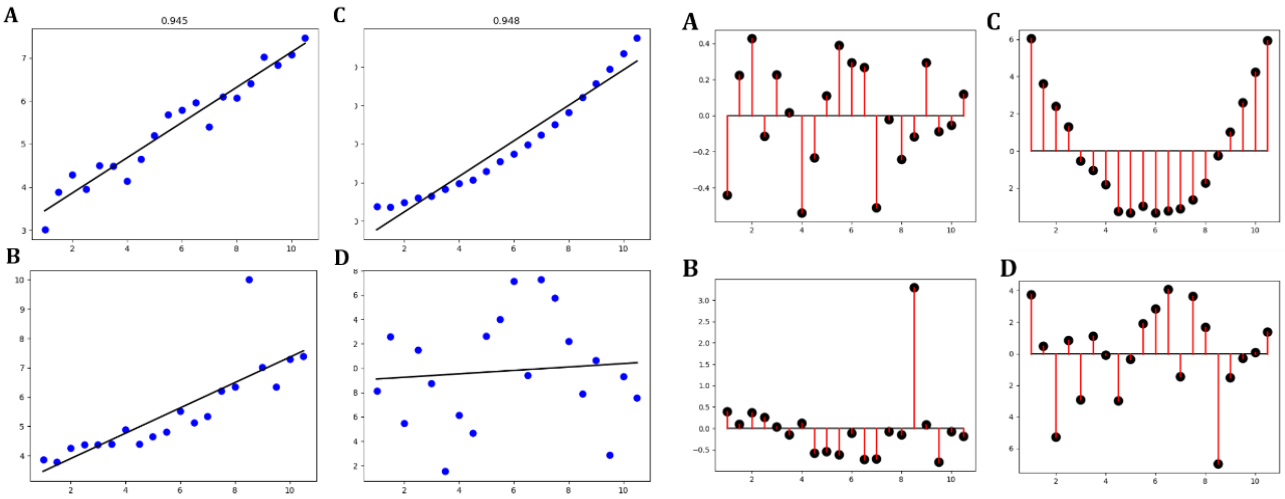


Figure 5.1.6 R-squared and residuals for different data sets

For example, a visual inspection of the plots, reveals that in plot (B), one of the measurements lay far from the common trend. Maybe because some error in the experiment or mistake while logging the data. This intractable point causes a deviation in the fitting values of the model. This point must be corrected or excluded from the data (possibly an outlier) before performing regression analysis.
On plot (C), although $R^2 = 0.948$, we see that there is no linear relationship between x and y. Hence, maybe a different theoretical model, such as second-degree polynomial may account more accurately for the given data.
On plot (D), the "jumps" in the measured values of y, for different x values suggest that probably there is no functional relation between these two quantities, and the regression line is meaningless.

5.1.7 Extension to non-linear models

Linear least square method can also be applied to other models by linearization of the models.
For example,

- Powers: $\boldsymbol{y = b\sqrt{x}}$ → Can be linearized as $\boldsymbol{y^2 = b^2 x}$
- Polynomials: $\boldsymbol{y = a + bx + cx^2 + dx^3}$ → This is just a case of using multiple regression since the equation is linear in the coefficients.
- Exponentials: $\boldsymbol{y = ae^{bx}}$ → Can be linearized as $\boldsymbol{ln(y) = ln(a) + b * x}$
- There are many other examples

As linear regression assumes that the model is linear and finds the parameters of the model, in nonlinear regression the model is a nonlinear function of the independent variable and contains different parameters. The nonlinear regression also uses the match criterion of a minimum quadratic error but causes the error to be minimized in another way, and it is called "Non-linear least square".
Unlike linear regression, there is no closed formula for calculating the optimal value of each of the parameters. Therefore, it uses an iterative algorithm that starts from "guessing" the parameter values, updates them according to their correspondence to the data and again checks the correspondence. After several iterations, the algorithm converges to a "stable" solution, thus finding the best fit for the data under the limitations of the current algorithm. The specific algorithm which is used (there are several possible algorithms for such calculations), determines the criterion deciding when the current solution is close enough to stop the iterations and receive the final fit of the model to the data. Hereby, different algorithms, may produce different fitting results. **This is the reason why non-linear regression is much less conclusive compared to the linear regression. We will always try to linearize our model!**